

Week 1 - Data foundations

Summary

This assignment introduces the first stage of a machine learning project for a city-wide bike sharing company. The goal is not yet to build a forecasting model, but to prepare the data that will support later machine learning work. You will combine data from multiple sources, explore and transform it, and organize the result as a reusable preprocessing pipeline. In doing so, you will gain experience with core data engineering and machine learning preparation tasks, using tools such as pandas, Jupyter notebooks, and Dagster.

Introduction

Our company operates a city-wide bike sharing service that gives customers two flexible ways to travel: they can either reserve a bike in advance or pick up one directly when it is available nearby. To keep this service reliable and efficient, the company must make sure that enough bikes are available in the right places at the right time. This creates an important planning challenge for the bike distribution department, which needs to estimate how many bikes will be required on the following day. To support this decision-making process, the company is starting a machine learning project that uses historical operational data to better understand demand patterns and improve daily bike allocation across the city.

In this first part of the project, the focus is not yet on building a forecasting model, but on preparing the data that will later be used for model training. Before a machine learning system can produce useful predictions, the underlying data must be explored, cleaned, structured, and transformed into a form that is suitable for analysis. In this assignment, you will work on the preprocessing steps needed to create a reliable dataset and make choices about how the data should be represented for later use in machine learning. This work forms the foundation for the forecasting task that will follow in the next stage of the project.

Goal

The goal of this assignment is to create a data preprocessing pipeline. In this pipeline, data is collected from the available sources, combined into one dataset, and prepared for later machine learning use. In this assignment, you will work toward building this pipeline step by step. Rather than treating preprocessing as a one-time activity, the pipeline defines a clear sequence of operations: data is loaded from the database tables and CSV files, transformed into a common structure, and prepared in a form that can be used in later stages of the machine learning project. This makes the workflow easier to understand, easier to maintain, and easier to repeat when new data becomes available.

A pipeline is especially useful because preprocessing is not only about writing code that works once, but about creating a process that can be run in a reliable and transparent way. Each step has clear inputs and outputs, and later steps depend on the results of earlier ones. This structure helps the team to keep track of how the final dataset is produced, and it reduces the risk of mistakes that often occur when preprocessing is done in an ad hoc way. In practice, such a pipeline can be started manually during development or executed on a schedule in a production setting, and its progress can be followed in a graphical interface that supports monitoring and observability. The result is a prepared dataset that forms the foundation for the machine learning training work that will follow later in the project.

Data sets

We now turn to the data that is available for this project. Data is at the heart of every machine learning project, and a clear understanding of the available sources is essential before we can start preparing it for analysis and later forecasting.

Booked and direct rentals

The first dataset contains records of individual **booked bike rentals** for each user and location. The data is extracted from the company's operational database and includes rentals that were reserved in advance and stored in a CSV file.

The second dataset contains records of individual **direct bike pickups** for each user and location. This data is also extracted from the company's operational database and includes rentals that were started without a prior booking.

Weather data

The third dataset contains the **weather data**. Because weather conditions strongly affect how many bikes are rented, the company extracted historical weather information from an external weather service and stored it in a CSV file. This dataset includes variables such as weather conditions, temperature, and humidity, which can serve as important explanatory features in the project.

Holiday calendar

The fourth dataset contains the **holiday calendar**. In addition to weather, holidays can also influence rental behavior because travel patterns often differ from regular working days. For that reason, the company manually created a CSV file listing all holidays in the project time frame, and this dataset can also be used as an exploratory feature when analyzing demand patterns.

Data preprocessing steps

In the following section, we outline the main preprocessing steps that have been identified for this project. Together, these steps describe how the available source data will be transformed into a dataset that can later be used in the machine learning workflow.

1. **Load and combine the operational data:** The first step is to load that was extracted from the company database. The data has to be transformed from individual records to aggregates per hour. Because rental activity is recorded in separate tables for booked rentals and direct pickups, both sources are needed to create a complete overview of bike usage. The output of this step is an hourly dataset containing the total number of rented bikes.
2. **Derive useful time-based information:** The second step is to derive additional information from the available date and timestamp fields. This is an example of *feature engineering*, where existing data is transformed into features that may help a machine learning model capture relevant patterns.
3. **Add the weather information:** The third step is to enrich the rental data with historical weather information. Since weather is expected to influence rental behavior, this source should be incorporated into the dataset.
4. **Add holiday information:** The fourth step is to include the holiday calendar in the dataset. Holidays may affect rental behavior and therefore provide an additional source of information for the project.

After these steps, the pipeline has produced the final dataset for this assignment.

How to approach this assignment?

A useful way to approach this assignment is to begin by setting up a solid Python development environment. We recommend using Visual Studio Code and configuring it for Python development. The dependencies needed for this assignment can then be installed based on the provided project specification.

When starting work on a data processing task, it is often helpful to begin in a notebook before building the full pipeline. A notebook is an interactive development environment in which you can write code in small parts, run it directly, inspect the results, and experiment with the data as you go. This makes it especially useful when you are still exploring the structure of the datasets and trying to understand which transformations are needed.

Once the main processing steps have become clear, they can be translated into a pipeline structure. At that point, the focus shifts from individual experiments to the overall data flow: which inputs are needed, which transformations take place, and which outputs are produced at each stage.

Tools & libraries

Here are suggestions for tools and Python libraries to complete your assignment:

- **Visual Studio Code:** Visual Studio Code will be used as the main development environment for this assignment. It provides a practical workspace for writing Python code, working with project files, running notebooks, and inspecting results in one place. For this assignment, the Python extension, the Jupyter extension, and Ruff as linter and formatter are especially useful.
- **uv:** The project uses **uv** for dependency management. This tool can be used to install the required packages from the provided project specification and to create and manage a virtual environment for the assignment. Working in a virtual environment helps keep the project dependencies isolated from other Python installations on your system, which makes the development setup cleaner and more reproducible.
- **Jupyter Notebooks:** A Jupyter notebook is an interactive environment for writing and running Python code in small parts, often called cells. Instead of writing a full script and running it from beginning to end, a notebook allows you to execute individual pieces of code, inspect intermediate results, and gradually build up your analysis. Jupyter notebooks are typically started as a local web server and then accessed through a browser, but they can also be created and run directly inside Visual Studio Code.
- **Pandas:** pandas is one of the standard libraries in the Python data ecosystem and has become a central tool for working with tabular data. It was developed to make data analysis and data manipulation in Python more practical and expressive, and it is now widely used in both industry and education.
Pandas can be used to read data in several formats and from different sources. Once the data is available in pandas, it can be inspected, filtered, transformed, combined, and prepared for later use in the preprocessing pipeline.
- **Dagster:** Dagster is a modern framework for building and managing data pipelines. It originated from the need to make data workflows easier to structure, understand, and operate, especially in settings where data processing involves multiple dependent steps and needs to be maintained over time.
A key idea in Dagster is that pipeline components can be defined as data assets with explicit dependencies between them. This makes it possible to describe how one output is produced from earlier inputs in a clear and structured way. In addition to implementation, Dagster is also powerful because it supports scheduled execution, observability, and monitoring through a graphical user interface. This makes it easier to understand how the pipeline runs and to inspect the state of the different processing steps.

Acceptance criteria

- The pipeline can be started successfully using the Dagster development server.
- All relevant assets in the pipeline materialize successfully without errors.
- The final asset writes the prepared dataset to a CSV file on disk.
- The codebase is logically structured according to the provided project template and follows a clear separation of responsibilities. For example, data assets should be implemented in their own Python files in the *assets* directory.
- Functions that implement pipeline logic, data loading, transformations, or asset definitions include docstrings in NumPy format. Very small helper functions may remain undocumented when their purpose is immediately clear from the code.
- The code is formatted and linted successfully with Ruff.

Tips

- Start-out in a Jupyter notebook. Only progress to Dagster when you have all data processing steps worked out and verified in the notebook.
- Dagster allows you to split up your code based on responsibilities. Use assets for data processing operations, resources for configuration and interacting with external systems (hint: your drive is also one!), and IO managers for data persistence operations.

Delivery format

Submit your work as a GitHub repository containing your project code. The final version of your assignment should be included in the latest commit on the *main* branch. Send the link to your repository to the track mentor on the evening before the code review.